HNSW Graph Meets Query Logs: Accelerating Dense Retrieval with Historical Information

Anonymous Author(s)

Abstract

Fast response time is critical for modern dense retrieval pipelines, where Approximate Nearest Neighbor (ANN) search over large vector collections often dominates query latency. Graph-based methods such as the Hierarchical Navigable Small World (Hnsw) are among the most widely adopted solutions. The efficiency of these methods depends on the quality of query routing: when the hierarchical traversal fails to reach a position close to the query's true nearest neighbors, the beam search at the ground level—which accounts for the vast majority of the total cost—must compensate with extensive exploration. In this paper, we draw inspiration from classical Information Retrieval — where query logs have long been exploited to improve efficiency — and introduce the Query Log Router (Qlr), a lightweight ANN index built on historical query vectors and a lookup table of precomputed results. At query time, the Qlr identifies past queries similar to the incoming one and uses their cached results to seed the search from a more informed position, significantly reducing the number of node expansions required. When no similar historical query is found, the system falls back to standard Hnsw search with minimal overhead. We further propose an adaptive mechanism that adjusts the search effort based on the confidence of the routing decision. Experiments on MsMarco-v1, MsMarco-v2, and a real-world query log across multiple embedding models show that Qlr achieves speedups of up to 2.6× while maintaining the same accuracy level.

1 Introduction

Information retrieval (IR) systems increasingly rely on neural models to capture semantic relationships between queries and documents. While these approaches substantially improve effectiveness over lexical matching, their computational cost poses significant challenges for scalable deployment. Early interaction-based models capture fine-grained query–document relationships, while transformer-based cross-encoders achieve state-of-the-art effectiveness at the cost of high inference complexity. To enable scalable search, representation-based approaches such as DPR [13], Contriever [9], MiniLM [21], Bge [2], and Snowflake [23] learn dense embeddings for queries and documents, enabling efficient similarity search in continuous vector spaces. Approximate Nearest Neighbour (ANN) search has thus become a core component of modern ad-hoc retrieval pipelines. ANN search methods have historically addressed the trade-off between retrieval accuracy and computational efficiency through space-partitioning techniques such as Inverted File Index (IVF) [12] and Locality-Sensitive Hashing (LSH) [7]. IVF index requires probing multiple inverted lists to achieve high recall, thereby increasing query-time computational cost. On the other hand, LSH-based approaches often require increasingly large index structures to maintain accuracy in high-dimensional spaces. Graph-based ANN methods overcome many of these limitations and have demonstrated strong trade-offs between effectiveness and efficiency. In this class of methods, the Hierarchical Navigable Small World (Hnsw) graph [14] has established itself as the dominant approach, enabling fast logarithmic-time traversal while maintaining high recall. Originally introduced by Malkov and Yashunin in 2018, Hnsw remains the most widely adopted proximity graph structure, owing to its ability to scale effectively with both dataset size and vector dimensionality [1]. Its versatility extends beyond dense vector search: Hnsw has been successfully adapted to sparse embeddings [4], attribute-filtered ANN search [18], and disk-based retrieval [10]. On the systems side, Hnsw serves as the backbone of major open-source ANN libraries such as FAISS [11], NMSLIB,[1] and kANNolo [4], as well as widely deployed vector databases including Qdrant, Weaviate, Pinecone, and Elasticsearch. This widespread adoption makes improvements to Hnsw's search efficiency immediately impactful across a broad range of research and production systems.

Hnsw organizes data points into a multi-layer graph: given a query, the search descends through progressively denser layers until it reaches the ground level, where a beam search—an iterative exploration that expands the most promising candidates by visiting their neighbors—identifies the nearest neighbors. The efficiency of Hnsw depends on the quality of its routing: the hierarchical traversal is designed to quickly guide the search toward the relevant region of the graph, but since the upper-level nodes are selected randomly, there is no guarantee that the entry point reached at the ground level will be close to the query's true nearest neighbors. When routing is ineffective, the beam search—which accounts for the vast majority of the total search cost—must explore a large portion of the graph to compensate, resulting in high latency. A growing line of research has begun to address this limitation, proposing probabilistic routing strategies that improve neighbor exploration to increase throughput without large accuracy losses, adaptive frameworks that formally analyze how entry point selection affects performance bounds, and learned candidate hubs or query-aware entry sets that accelerate graph traversal while preserving result quality [16, 17, 19, 24].

In this paper, we take a page from classical Information Retrieval, where query logs have long been used to improve search efficiency [20], and apply this idea to the domain of graph-based ANN search. We introduce the Query Log Router (Qlr), a lightweight auxiliary structure that leverages historical queries to select better entry points for graph traversal. The Qlr is itself an ANN index, built over a sample of historical query vectors and paired with a look-up table that maps each indexed query to its previously computed nearest neighbors in the document collection. At search time, rather than relying on Hnsw's hierarchy to reach a starting point that may still be far from the query's nearest neighbors, the Qlr identifies past queries similar to the incoming one and reuses their cached neighbors to seed the beam search from a more informed position. When

[1] https://github.com/nmslib/nmslib

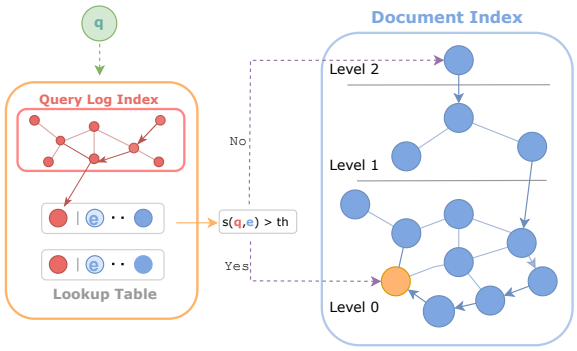

Figure 1: High-level view of our Hnsw-based ANN search architecture exploiting Qlr.

no sufficiently similar past query is found, the system seamlessly falls back to standard Hnsw search with minimal overhead. We further propose an adaptive mechanism that dynamically adjusts the search effort based on the confidence of the routing decision.

Our experimental evaluation on two widely adopted benchmarks for dense retrieval, i.e., MsMarco-v1 and MsMarco-v2, shows that Query Log Router accelerates search by up to 1.8× on MsMarco-v1 and up to 2.6× on MsMarco-v2 at the same accuracy level. As neither MsMarco-v1 nor MsMarco-v2 provides query-frequency information (i.e., whether a query is popular), we further complement our evaluation with queries in a real-world MSN query log. Using MsMarco-v1 as the document collection and the most frequent queries in the query log to build the Qlr, we show that our technique yields speedups of up to 2.3× at the same accuracy level.

2 Methodology

Given a query vector $q \in \mathbb{R}^d$ and a collection of embeddings D in the same space, we aim to efficiently retrieve the approximate top-k nearest neighbors of q using a graph-based ANN index $\mathcal{I}_D$ built over D. In the following, we describe how retrieval is performed in an Hnsw graph and how our Query Log Router framework can be exploited to speedup the search mechanism.

2.1 Hnsw

The Hnsw graph builds a multi-layer proximity graph over the embedding dataset D, where spatially close vectors are connected by edges to enable efficient navigation toward the query's neighborhood. Each level of the graph contains a progressively sparser subset of the vectors from the level below, with the ground level (level 0) containing the entire dataset. This hierarchical organization accelerates traversal: the sparser upper layers provide long-range connections that can quickly reach the relevant region of the space, while the ground level enables fine-grained local refinement. Given a query vector q, the search begins at the topmost layer from a fixed entry point that is common to all queries. Routing proceeds greedily: at each step, the algorithm evaluates the distances between q and the neighbors of the current node, and moves to the closest one. This process repeats until a local minimum is reached, i.e., no neighbor is closer to q than the current node. The search then descends to the next layer, using the current node as the entry point for the new level. Once the ground level is reached, the algorithm switches to a beam search that maintains a dynamic candidate set of size `ef-search` rather than tracking a single node. At each step, the closest unexplored candidate is expanded by evaluating its neighbors, and the candidate set is updated accordingly. The search terminates when all candidates have been explored or cannot improve the current top-k results, which are then returned.

2.2 Query Log Router

As described above, Hnsw starts every search from a fixed entry point, navigating through the hierarchy before reaching the base level. In this section, we introduce our Query Log Router (Qlr) index, which exploits historical queries to quickly route the current query towards the *hot zone* of the bottom-layer graph, namely the zone where the closest vectors are likely to be located. Our approach builds on a simple observation: similar queries tend to retrieve similar results. If a new query closely resembles one that has been previously processed, we can reuse the latter's results to initialize the beam search at the ground level of Hnsw from a position already close to the query's nearest neighbors. Since the beam search at ground level accounts for the majority of the total search cost, starting from a better entry position substantially reduces the number of nodes to be explored at that level, thereby lowering the overall search cost.

The Query Log Router (Qlr) is a lightweight auxiliary structure that sits in front of the standard Hnsw index. The system we propose comprises three components (Figure 1):

- A *query log index* $\mathcal{I}_Q$, built over a selection of historical query vectors, used to quickly identify past queries similar to the incoming one.
- A *lookup table* EP (Entry Points), that maps each historical query in I_Q to the precomputed list of its closest embeddings in D.
- The *document index* $\mathcal{I}_D$, i.e., the standard Hnsw graph over the collection of embeddings D.

Construction Phase. Qlr is built from a document collection D and a set of queries Q_L extracted from a historical query log, where $|Q_L| \ll |D|$. Two separate indexes are first constructed: a *document index* $\mathcal{I}_D$ over D and a *query log index* $\mathcal{I}_Q$ over Q_L. Both can be instantiated using any ANN indexing method; in our implementation, we use Hnsw graph indexes for both. The lookup table is populated at construction time as follows. For each historical query $q_l \in Q_L$, we search $\mathcal{I}_D$ to retrieve its k_{ep} nearest neighbors in D. These results are stored in the entry $\text{EP}(q_l)$ of the lookup table and can later be used to seed beam search for similar incoming queries.

Search Phase. When a new query q arrives, it is first searched in the query log index $\mathcal{I}_Q$ to retrieve the k' most similar historical queries (line 1 in Alg. 1). Let s denote the similarity (higher is better) between q and the closest retrieved historical query. If $s < th$, where th is a predefined threshold, the historical queries in Q_L are deemed insufficiently similar, and the system falls back to standard Hnsw search on $\mathcal{I}_D$ (line 4 in Alg. 1). Otherwise, for each retrieved log query q_l, the precomputed entry points $\text{EP}(q_l)$ are collected from the lookup table and merged into a single candidate set C (line 5

Algorithm 1: Query processing with QLR.

Input: q: query; $\mathcal{I}_Q$: query log index; $\mathcal{I}_D$: document index; k: number of results to return; k': number of historical queries to retrieve from $\mathcal{I}_Q$; th: similarity threshold; $s_{\max}$: reference similarity; ef: default beam width; $\texttt{ef}_{\min}$: minimum beam width; EP: look-up table.

Result: A set with the top-k nearest neighbors of q.

1: $\{(q_1, s_1), \ldots, (q_{k'}, s_{k'})\} \leftarrow \textsc{Search}(\mathcal{I}_Q, q, k')$
2: $s \leftarrow s_1$
3: **if** $s < th$ **then**
4: **return** $\textsc{HnswSearch}(\mathcal{I}_D, q, k, \texttt{ef})$
5: $C \leftarrow \bigcup_{i=1}^{k'} \mathrm{EP}(q_i)$
6: **if** $s > s_{\max}$ **then**
7: $\texttt{ef}' \leftarrow \texttt{ef}_{\min}$
8: **else**
9: $\texttt{ef}' \leftarrow \texttt{ef}_{\min} + (\texttt{ef} - \texttt{ef}_{\min}) \cdot \frac{s_{\max} - s}{s_{\max} - th}$
10: **return** $\textsc{BeamSearch}(\mathcal{I}_D, q, k, \texttt{ef}', C)$

in Alg. 1). This set is then used to initialize the beam search at the ground level of $\mathcal{I}_D$, allowing it to converge with significantly fewer node expansions (line 10 in Alg. 1).

Adaptive Search Depth. The similarity s between the incoming query and its closest historical query carries more information than a binary match signal: a higher similarity suggests that the precomputed entry points are likely to land closer to the true nearest neighbors, and thus less exploration is needed. We exploit this observation to dynamically adapt the `ef_search` parameter, which controls the size of the candidate set during beam search (lines 6-9 in Alg. 1). At index construction time, we compute $s_{\max}$ as the 75th percentile of the similarities between each historical query $q_l \in Q_L$ and its nearest neighbor in D_I. At query time, `ef_search` is adjusted as follows:

- If $s < th$, the query is not routed and `ef_search` is left at its default value.
- If $s > s_{\max}$, the match is highly confident and `ef_search` is set to a minimum value $\texttt{ef}_{\min}$.
- Otherwise, `ef_search` is linearly interpolated between its default value and $\texttt{ef}_{\min}$ according to $\frac{s_{\max} - s}{s_{\max} - th}$.

Practical Considerations. A key design requirement of QLR is that querying the query log index $\mathcal{I}_Q$ must introduce minimal latency overhead. This is particularly important in the negative case: as shown in Algorithm 1, when no sufficiently similar historical query is found ($s < th$), the system falls back to standard HNSW search, and the entire cost of querying $\mathcal{I}_Q$ is wasted.

Although $|Q_L| \ll |D|$, searching an HNSW index built over full-dimensional query vectors in $\mathbb{R}^d$ incurs non-negligible cost. On the MSMARCO-V1 dataset, encoded with the SNOWFLAKE encoder, with $|Q_L| \approx \frac{1}{10}|D|$, the average latency of searching $\mathcal{I}_Q$ at full dimensionality is 554 μs, compared to 726 μs for a search on $\mathcal{I}_D$, at the the same level of approximation.

We mitigate this cost by applying Principal Component Analysis (PCA) to reduce the dimensionality of the vectors in Q_L before constructing $\mathcal{I}_Q$. Crucially, the task performed by $\mathcal{I}_Q$ is not a standard ANN retrieval problem: it does not require high-recall identification of the nearest historical queries, but rather a binary determination of whether any historical query within a similarity radius th of the incoming query exists. This relaxed requirement makes the search tolerant to the approximation introduced by dimensionality reduction. We experimentally verify that with PCA we can effectively search the query log in less than 250 μs. In practice, the PCA projection matrix is computed on Q_L at construction time and stored alongside $\mathcal{I}_Q$. At query time, the incoming query is projected into the reduced space via a fast matrix multiplication, whose cost is negligible relative to the graph traversal and can be efficiently amortized over batches of queries. As an example, with batch size 32, the transformation from 1024 dimension to 256 takes less than 20 μs per query. We leave the exploration of other compression techniques that accelerate search, e.g., binarization or Locality Sensitive Hashing [7], for future work.

3 Experiments

Datasets. We conduct our experimental evaluation on two publicly available datasets, MSMARCO-V1 and MSMARCO-V2, and the Microsoft Data Asset (MDA) query log, which we describe below.

- MSMARCO-V1 is a widely used benchmark for dense retrieval consisting of a collection of 8.8M passages. In our evaluation, we use the `dev.small` split as the test set, which comprises 6,980 queries. We use the `train` split, containing 800k queries, as the source of historical queries in Q_L. We use MSMARCO-V1 as the document collection.
- MSMARCO-V2 is an extended version of MSMARCO-V1 containing approximately 138M passages. We use the `dev1` split, comprising 3,903 queries, as the test set, and again use the `train` split of MSMARCO-V1 to construct Q_L.
- Microsoft Data Asset (MDA) [3]. We use an excerpt of the MSN Search query log released for the WSCD workshop at WSDM 2009. The dataset contains 15 million queries from U.S. users, sampled over one month in 2006. This real-world query log enables us to evaluate the effectiveness of exploiting historical query distributions when selecting queries stored in the QUERY LOG ROUTER index. Queries are sorted chronologically, and the last 10,000 are used as a test set. The historical query set Q_L is extracted from the remaining part of the query log by considering the 1,483,055 queries occurring at least two times.

Encoders. In our experiments, we employ the following encoders.

- MINILM [22]: lightweight 384-dim cross-encoder for general semantic search, achieving 30.84 of MRR@10 on MSMARCO-V1.
- CONTRIEVER [8]: encoder which uses BERT as a backbone, producing embeddings of size 768. It achieves 34.06 of MRR@10 on MSMARCO-V1.
- SNOWFLAKE [23]: large (1024-dim) embedding model optimized for retrieval, MTEB [15] leader with strong long-context handling. It yields 38.21 of MRR@10 on MSMARCO-V1 and 16.58 of MRR@10 on MSMARCO-V2.

Metrics. We evaluate QLR against standard HNSW along the efficiency-effectiveness tradeoff. Efficiency is measured as average query latency in microseconds (μs), while effectiveness is measured as Accuracy@10, defined as the fraction of the true 10 nearest neighbors appearing among the top-10 retrieved results, averaged over all queries. The ground truth is obtained by exhaustive search over

Table 1: Comparison in terms of latency (μs) vs Accuracy@10 on MsMarco-v1 and MsMarco-v2.

		Accuracy@10 (%)				
		93	95	97	98	99
MsMarco-v1						
MiniLM	Hnsw (Faiss)	225 1.3×	265 1.4×	305 1.3×	382 1.3×	608 1.3×
	Hnsw (kANNolo)	203 1.2×	245 1.3×	309 1.3×	385 1.3×	619 1.4×
	Qlr	**169** –	**193** –	**240** –	**300** –	**454** –
Contriever	Hnsw (Faiss)	309 1.0×	399 1.1×	575 1.3×	830 1.4×	1872 1.3×
	Hnsw (kANNolo)	358 1.2×	444 1.3×	651 1.4×	948 1.6×	2041 1.4×
	Qlr	**302** –	**350** –	**457** –	**609** –	**1432** –
Snowflake	Hnsw (Faiss)	476 1.3×	547 1.4×	751 1.6×	1084 1.8×	1926 1.8×
	Hnsw (kANNolo)	413 1.2×	512 1.3×	726 1.5×	940 1.5×	1543 1.5×
	Qlr	**359** –	**391** –	**483** –	**610** –	**1050** –
MsMarco-v2						
Snowflake	Hnsw (kANNolo)	**663** 0.7×	**995** 1.0×	3394 2.6×	-	
	Qlr	898 –	1000 –	**1289** –	**2876** –	

the full dataset. For each target Accuracy@10 level, we report the latency of the fastest configuration (i.e., the combination of hyper-parameters; see below for more details) that achieves it, allowing a fair comparison across methods.

Hyperparameters. For MiniLM, Contriever, and Snowflake with Hnsw we build the index with $m = 32$, `ef-construction` = 500 and with tested `ef-search` in [10, 200] step 10. For Qlr, we use PCA to reduce the query embeddings to 1/4 of the original size, so 96, 128, 256 for MiniLM, Contriever, and Snowflake, respectively. We tested $k_{ep} = 10$, $k' \in \{10, 20\}$ and $th \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$ for L_2 normalized embeddings (MiniLM, Snowflake) and $th \in \{1.2, 1.3, 1.4, 1.5, 1.6\}$ for Contriever which is not normalized.

Implementation. We employ the Hnsw implementation made available by the kANNolo [5] library (Rust version `1.92.0-nightly`), and we implement Qlr on top of it. For reference, in our experiments on MsMarco-v1, we include the performance of the Hnsw algorithm as implemented in the widely adopted Faiss library [6].[2] Our code will be made available upon publication.

Hardware Platform. We conduct our experiments on a machine equipped with four Intel Xeon Gold 6252N CPUs (2.30 GHz), with a total of 192 cores (96 physical and 96 hyper-threaded) and 1 TiB of RAM. The index construction is done using all the available threads, while searching the index is done with only one.

3.1 Results

MsMarco-v1. Table 1 reports the latency-accuracy tradeoff on MsMarco-v1. Qlr consistently outperforms both Hnsw baselines (Faiss and kANNolo implementation) across all encoders and accuracy targets, achieving speedups of up to 1.8× over Faiss and 1.5× over the kANNolo Hnsw implementation. Notably, the speedup tends to grow as the accuracy target increases, where standard Hnsw becomes increasingly expensive to satisfy high-accuracy constraints. Table 1 also confirms that kANNolo Hnsw implementation is competitive with Faiss: it is faster on MiniLM and Snowflake, and slightly slower on Contriever. We therefore omit Faiss from the remaining experiments on MsMarco-v2 and Mda.

[2]https://github.com/facebookresearch/faiss

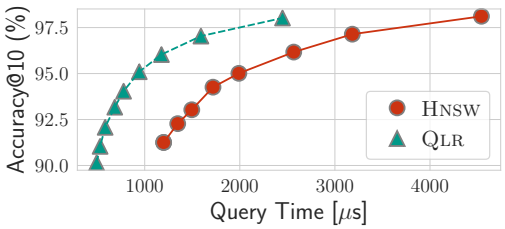

Figure 2: Latency/Accuracy tradeoffs of Hnsw and Qlr using the Mda query log and MsMarco-v1 as document collection.

MsMarco-v2. Table 1 also reports results on MsMarco-v2 using the Snowflake encoder. While at lower accuracy targets ($\lesssim 95\%$), Qlr offers no advantage over Hnsw, it achieves up to 2.6× speedup and, crucially, can reach accuracy levels that Hnsw fails to attain entirely within reasonable latency budgets, largely surpassing the speedup achieved on MsMarco-v1. We investigate this behavior further. At `ef-search` = 400, standard Hnsw achieves 97% Accuracy@10, yet a small subset of queries (roughly 50) has 0% recall—these are hard queries for which Hnsw consistently fails to retrieve any true neighbor. Qlr reaches 98% Accuracy@10 at lower latency, because many of these hard queries have close counterparts in the training set Q_L, allowing the search to be initialized from highly relevant regions of the index. The benefit mainly emerges when the test query is either a reformulation or is semantically related to one of the historical queries in Q_L (e.g., *"upstart meaning"* reformulated in *"what is an upstart"*; *"what is compressed air"* benefiting from historical query *"what is compressed oxygen"*).

MsMarco-v1 with Mda query log. Figure 2 reports the latency-accuracy tradeoff employing Qlr, the Mda query log for Qlr and the MsMarco-v1 dataset as the document collection. Qlr consistently outperforms Hnsw (kANNolo) across all accuracy targets, with speedups ranging from 2.3× at 91% accuracy to 1.9× at 98%. Recall that on MsMarco-v1 using the `train` set of queries as query log, the maximum achieved speedup on Snowflake (Table 1) was 1.5×, which means that introducing the query frequency information can consistently boost the performance.

4 Conclusions and Future Work

In this paper, we presented the Query Log Router (Qlr), a light-weight auxiliary structure designed to accelerate dense retrieval by exploiting historical query information within ANN graph-based indexes. It efficiently identifies past queries similar to an incoming one and leverages their cached results to seed the ground-level beam search. QLR achieves substantial speedups of up to 1.8× on MsMarco-v1, 2.6× on MsMarco-v2, and 2.3× when exploiting the real-world Mda query distribution. Notably, the framework excels at resolving "hard queries" by leveraging semantic similarities found in historical data. By successfully bridging classical query log mining with modern proximity graphs, QLR provides a scalable and efficient path forward for high-performance dense retrieval pipelines.

References

[1] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2025. Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. *Proc. ACM Manag. Data* 3, 1, Article 43 (Feb. 2025), 31 pages. doi:10.1145/3709693

[2] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216* 4, 5 (2024).

[3] Nick Craswell, Rosie Jones, Georges Dupret, and Evelyne Viegas (Eds.). 2009. *Proceedings of the 2009 workshop on Web Search Click Data, WSCD@WSDM 2009, Barcelona, Spain, February 9, 2009*. ACM. doi:10.1145/1507509

[4] Leonardo Delfino, Domenico Erriquez, Silvio Martinico, Franco Maria Nardini, Cosimo Rulli, and Rossano Venturini. 2025. kANNolo: Sweet and Smooth Approximate k-Nearest Neighbors Search. In *Advances in Information Retrieval: 47th European Conference on Information Retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, Proceedings, Part IV* (Lucca, Italy). Springer-Verlag, Berlin, Heidelberg, 400–406. doi:10.1007/978-3-031-88717-8_29

[5] Leonardo Delfino, Domenico Erriquez, Silvio Martinico, Franco Maria Nardini, Cosimo Rulli, and Rossano Venturini. 2025. kANNolo: Sweet and Smooth Approximate k-Nearest Neighbors Search. In *Proceedings of the 47th European Conference on Information Retrieval*. 400–406.

[6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]

[7] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing* (Dallas, Texas, USA) *(STOC '98)*. Association for Computing Machinery, New York, NY, USA, 604–613. doi:10.1145/276698.276876

[8] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Towards unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118* 2, 3 (2021).

[9] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. Unsupervised Dense Information Retrieval with Contrastive Learning. *Trans. Mach. Learn. Res.* 2022 (2022). http://dblp.uni-trier.de/db/journals/tmlr/tmlr2022.html#IzacardCHRBJG22

[10] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf

[11] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572

[12] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57

[13] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 6769–6781. doi:10.18653/v1/2020.emnlp-main.550

[14] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473

[15] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. Mteb: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2014–2037.

[16] Cristina Ioana Muntean, Franco Maria Nardini, Raffaele Perego, Guido Rocchietti, and Cosimo Rulli. 2025. Efficient Conversational Search via Topical Locality in Dense Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2749–2753.

[17] Yutaro Oguri and Yusuke Matsui. 2024. Theoretical and Empirical Analysis of Adaptive Entry Point Selection for Graph-based Approximate Nearest Neighbor Search. *CoRR* abs/2402.04713 (2024). https://doi.org/10.48550/arXiv.2402.04713

[18] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[19] Jiancheng Ruan, Tingyang Chen, Renchi Yang, Xiangyu Ke, and Yunjun Gao. 2025. Empowering Graph-based Approximate Nearest Neighbor Search with Adaptive Awareness Capabilities. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) *(KDD '25)*. Association for Computing Machinery, New York, NY, USA, 2444–2454. doi:10.1145/3711896.3736930

[20] Fabrizio Silvestri. 2010. Mining Query Logs: Turning Search Usage Data into Knowledge. *Found. Trends Inf. Retr.* 4, 1-2 (2010), 1–174. doi:10.1561/1500000013

[21] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 5776–5788. https://proceedings.neurips.cc/paper_files/paper/2020/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[22] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in neural information processing systems* 33 (2020), 5776–5788.

[23] Puxuan Yu, Luke Merrick, Gaurav Nuti, and Daniel Campos. 2024. Arctic-Embed 2.0: Multilingual Retrieval Without Compromise. arXiv:2412.04506 [cs.CL] https://arxiv.org/abs/2412.04506

[24] Chao Zhang and Renée J. Miller. 2025. Distribution-Aware Exploration for Adaptive HNSW Search. arXiv:2512.06636 [cs.DB] https://arxiv.org/abs/2512.06636